HYPER-V IOPS MONITOR

Complete Technical Documentation

Version 1.0 | April 2026 | Internal Use Only

Purpose	Adaptive IOPS monitoring for Hyper-V virtual machines with automated Discord alerting
Scope	Applies to all Windows Server hosts running the Hyper-V role
Audience	System administrators, infrastructure engineers, and IT operations teams
Status	Production ready

Table of Contents

1. Executive Summary

The Hyper-V IOPS Monitor is a pair of PowerShell scripts designed to track disk I/O activity across all virtual machines running on a Hyper-V host, and to send automated alerts to a Discord channel when any VM is consuming an abnormally high amount of disk resources.

What makes this solution different from conventional monitoring tools is its adaptive approach. Rather than applying a single, fixed IOPS threshold to every virtual machine — which is inherently unfair and produces both false alarms and missed alerts — this system first learns the normal behaviour of each VM individually, and then raises an alert only when a VM exceeds a configurable percentage of its own learned peak. This means that a small web server and a large database server are each judged against their own normal workload, not a one-size-fits-all number.

Key Capabilities

Adaptive per-VM thresholds • Automatic peak learning • Discord rich-embed alerts • IPv4-forced outbound connectivity • Persistent Windows Scheduled Task • Structured log file with full audit trail

2. Installation Script — Install-IOPSMonitorTask.ps1

This script is a one-time setup utility. Its only job is to register the monitoring script as a Windows Scheduled Task so that it starts automatically every time the server boots. It also provides commands to check the status of the task or remove it.

2.1 Actions

- Install — Validates inputs, creates the scheduled task, and starts it immediately.
- Status — Displays the current state of the task, including the last run time and result.
- Remove — Unregisters and deletes the scheduled task.

2.2 Parameters

Parameter	Required	Default	Description
`-Action`	Yes	—	One of: Install, Remove, or Status
`-DiscordWebhookUrl`	Yes*	—	Your Discord channel webhook URL (* required for Install only)
`-ThresholdPercent`	No	80	Percentage of each VM's learned peak IOPS that triggers an alert (10–99)

Parameter	Required	Default	Description
`-BaselineDurationMinutes`	No	10	Number of minutes the script spends learning each VM's peak before monitoring begins
`-SampleIntervalSeconds`	No	30	How frequently (in seconds) IOPS counters are polled
`-AlertCooldownMinutes`	No	5	Minimum time (in minutes) between repeat alerts for the same VM
`-ScriptPath`	No	Same folder	Full path to Watch-HyperVIOPS.ps1 if it is not in the same directory as this script

2.3 What the Install Action Does

1. Validates that the webhook URL is present and that Watch-HyperVIOPS.ps1 exists at the specified path.
2. Constructs the full PowerShell command line, passing all configuration parameters through to the monitoring script.
3. Registers a Windows Scheduled Task configured to:
   - Run under the SYSTEM account with the highest privilege level.
   - Start automatically at every system boot.
   - Restart up to five times on failure, with a two-minute delay between attempts.
   - Run indefinitely with no execution time limit.
4. Starts the task immediately so monitoring begins without requiring a reboot.
5. Confirms the task state in the console output.

3. Monitoring Script — Watch-HyperVIOPS.ps1

This is the core engine that runs continuously in the background. It performs the baseline learning phase, transitions to active monitoring, and dispatches Discord alerts when thresholds are exceeded.

3.1 Parameters

Parameter	Required	Default	Description
`-DiscordWebhookUrl`	Yes	—	Discord webhook URL (validated on startup)
`-ThresholdPercent`	No	80	Percentage of each VM's peak IOPS that triggers an alert
`-BaselineDurationMinutes`	No	10	Duration of the initial learning window in minutes
`-SampleIntervalSeconds`	No	30	Polling interval in seconds
`-AlertCooldownMinutes`	No	5	Cooldown between repeat alerts for the same VM
`-LogPath`	No	C:\Scripts\HyperV-IOPS-Monitor.log	Path to the rolling log file

3.2 Core Functions

Resolve-IPv4

Resolves the hostname discord.com to its IPv4 address before making the webhook request. This is necessary because some Windows Server environments route DNS queries to IPv6 addresses by default, which can result in connection timeouts if IPv6 is not fully supported on the network path. By explicitly targeting an IPv4 address and passing the correct Host header, the script ensures reliable delivery regardless of the server's IPv6 configuration.

Write-Log

Writes a timestamped, colour-coded entry to both the console and the log file. Entries are classified as INFO (routine), WARN (threshold exceeded or suppressed alerts), or ERROR (failures such as counter read errors or failed webhook calls).

Send-DiscordAlert

Constructs the Discord embed payload as a raw JSON string and sends it via the webhook URL. Building the JSON directly — rather than using PowerShell's ConvertTo-Json — avoids a known issue

where boolean values in nested hashtables can be serialised incorrectly, causing Discord to reject the request silently. The function also logs the HTTP status code and Discord's error response body in the event of a failure, making diagnosis straightforward.

Get-VMIOPSCounters

Queries the Hyper-V Virtual Storage Device Windows Performance Counter set to retrieve per-virtual-disk IOPS and throughput figures. Because performance counter instance names use internal VM GUIDs rather than friendly names, the function calls Get-VM to build a GUID-to-name lookup table. Read and write metrics from all virtual disks belonging to the same VM are then aggregated to produce a single per-VM total.

3.3 Baseline Phase

The baseline phase runs for the duration specified by -BaselineDurationMinutes. During this period the script polls counters on the normal -SampleIntervalSeconds interval and records the highest total IOPS observed for each VM. No alerts are generated.

When the window closes, any VM whose recorded peak is below 100 IOPS has its peak raised to that floor value. This prevents false positives for VMs that happened to be idle or lightly loaded at startup.

The script then logs the complete set of learned peaks and the corresponding absolute IOPS values at which each VM will begin generating alerts before transitioning to monitoring mode.

3.4 Monitoring Phase

The monitoring phase runs as an infinite loop. On each iteration the script:

6. Polls IOPS counters for all running VMs.
7. Checks whether any VM has set a new personal peak and updates the stored value if so.
8. Detects VMs that were not present during baseline (e.g., started after the server booted) and assigns them an initial peak equal to their current IOPS, subject to the 100 IOPS floor.
9. Calculates each VM's usage percentage as: current IOPS divided by peak IOPS, multiplied by 100.
10. If the usage percentage meets or exceeds ThresholdPercent, checks whether the per-VM cooldown period has elapsed.
11. If the cooldown has passed, dispatches a Discord alert and records the current time as the last alert timestamp for that VM.
12. If the cooldown has not passed, logs a suppressed-alert entry noting when the next alert will be permitted.
13. Logs all per-VM statistics to the console and log file regardless of whether an alert was triggered.

14. Sleeps for SampleIntervalSeconds before beginning the next iteration.

3.5 Error Handling

- Performance counter read failures are caught and logged. The loop continues rather than terminating.
- Discord webhook failures are caught, logged with HTTP status code and response body, and do not interrupt monitoring.
- An outer try/catch in the main loop ensures that any unexpected error is logged and the loop resumes rather than the script exiting.
- The scheduled task is configured to restart the script automatically if it does terminate for any reason.

4. Deployment Guide

4.1 Prerequisites

- Windows Server with the Hyper-V role installed and running.
- PowerShell 5.1 or higher (must be run as Administrator).
- Both script files (Install-IOPSMonitorTask.ps1 and Watch-HyperVIOPS.ps1) copied to C:\Scripts\ on the Hyper-V host. This folder must be accessible by the SYSTEM account.
- Discord webhook URL obtained from your Discord server channel (Channel Settings > Integrations > Webhooks > New Webhook). This is required for alert delivery.
- Windows Task Scheduler service running. The installation script will automatically create and configure a scheduled task to start the monitor at every system boot.

4.2 Step-by-Step Installation

15. **Copy both scripts to the host server**

Place Install-IOPSMonitorTask.ps1 and Watch-HyperVIOPS.ps1 in the same folder, for example C:\Scripts\.

16. **Create a Discord webhook**

In your Discord server, go to the channel where you want alerts to appear. Open Channel Settings, navigate to Integrations, click Webhooks, then New Webhook. Copy the generated URL.

17. **Open PowerShell as Administrator**

```
cd C:\Scripts
```

18. **Run the installation command**

The example below uses the recommended defaults. Adjust the parameters to suit your environment. Plain-English explanations of each setting are provided in the table beneath the command.

```
.\Install-IOPSMonitorTask.ps1 `
    -Action Install `
    -DiscordWebhookUrl "https://discord.com/api/webhooks/YOUR_ID/YOUR_TOKEN" `
    -ThresholdPercent 80 `
    -BaselineDurationMinutes 10 `
    -SampleIntervalSeconds 30 `
    -AlertCooldownMinutes 5
```

Setting	Value	What it means in plain English
`-ThresholdPercent 80`	80%	Send an alert when a VM is using 80% or more of the highest IOPS it has ever been seen reaching. Lower this number to alert earlier; raise it to only alert in more extreme situations.

Setting	Value	What it means in plain English
`-BaselineDurationMinutes 10`	10 min	Wait 10 minutes at startup to watch and learn each VM's normal peak IOPS before sending any alerts. Increase this if your VMs take longer to reach their typical workload after a reboot.
`-SampleIntervalSeconds 30`	30 sec	Check every VM's disk activity every 30 seconds. Lower this for faster detection; raising it reduces the minor background load of the script.
`-AlertCooldownMinutes 1`	5 min	Once an alert has been sent for a specific VM, wait at least 5 minute before sending another alert for that same VM. Increase this if your Discord channel is receiving too many repeated notifications.

19. **Verify the task is running**

```
.\Install-IOPSMonitorTask.ps1 -Action Status
```

20. **Monitor the log file**

```
Get-Content C:\Scripts\HyperV-IOPS-Monitor.log -Wait -Tail 30
```

4.3 Removing the Monitor

```
.\Install-IOPSMonitorTask.ps1 -Action Remove
```

This unregisters the scheduled task. The script files themselves are not deleted and can be reinstalled at any time.

5. Logging and Troubleshooting

5.1 Log File

All script activity is written to a plain-text log file at C:\Scripts\HyperV-IOPS-Monitor.log (configurable via -LogPath). Each entry follows the format:

```
[yyyy-MM-dd HH:mm:ss] [LEVEL] message
```

- [INFO] — Routine operational data: per-VM IOPS readings, peak updates, Discord confirmations.
- [WARN] — Threshold crossings and suppressed alerts.
- [ERROR] — Failures such as counter read errors or webhook delivery failures, including HTTP status codes and response bodies.

5.2 Common Issues and Resolutions

Symptom	Likely Cause	Recommended Action
Task installed but no log file appears	Incorrect script path or SYSTEM account cannot access the folder	Verify that Watch-HyperVIOPS.ps1 exists at the path registered in the task. Run Get-ScheduledTask -TaskName HyperV-IOPS-Monitor | Select -ExpandProperty Actions | Select -ExpandProperty Arguments to inspect the registered command.
Get-Counter failed messages in the log	Hyper-V performance counters not available	Confirm the Hyper-V role is installed: Get-WindowsFeature Hyper-V. Counters are only available when at least one VM has been created.
Discord alerts timing out	SYSTEM account cannot reach discord.com, often due to IPv6 routing	Run Test-NetConnection -ComputerName discord.com -Port 443. If TcpTestSucceeded is False, check firewall rules for outbound HTTPS. The script attempts IPv4 resolution automatically.
Alerts firing immediately after baseline	All VMs were idle during the learning window, causing very low stored peaks	Increase -BaselineDurationMinutes to cover the period when VMs are under normal load, or allow the script to run for a full working day so peaks adjust upward organically.
No alerts despite high disk activity	Threshold set too high, or VM peak has grown and the threshold has scaled with it	Check the log for per-VM IOPS readings and current usage percentages. Reduce -ThresholdPercent if appropriate.

Symptom	Likely Cause	Recommended Action
Script stops after a period	Unhandled exception terminated the process	The scheduled task will restart the script within two minutes. Examine the log file for the last ERROR entry to identify the root cause.

6. Security and Performance Considerations

6.1 Security

- The scheduled task runs as the built-in SYSTEM account, which has the permissions necessary to read Hyper-V performance counters without requiring a named service account.
- The Discord webhook URL is stored in the scheduled task's command-line arguments and is therefore visible to any local administrator. For environments with stricter security requirements, the script can be modified to read the URL from Windows Credential Manager or an encrypted file at startup.
- TLS 1.2 is enforced for all outbound HTTPS connections. Earlier protocol versions are not negotiated.
- The script makes no inbound network connections and does not open any listening ports.

6.2 Performance Impact

- The default 30-second polling interval reads a small set of Windows Performance Counters on each cycle. The overhead is negligible on any modern server-class hardware.
- The script spends the vast majority of its time in a sleep state. CPU consumption in steady-state monitoring is effectively zero.
- Memory usage remains low and stable. The script does not accumulate historical data and does not grow in memory over time.

7. Complete Process Flow

The diagram below illustrates the full execution path of the monitoring script from initial startup through to steady-state alert processing.

START — Server boots
Scheduled Task launches Watch-HyperVIOPS.ps1
BASELINE PHASE — Runs for BaselineDurationMinutes
Poll IOPS every SampleIntervalSeconds
Record highest IOPS per VM
Apply 100 IOPS minimum floor to any idle VM
Log learned peaks and alert thresholds
MONITORING PHASE — Infinite loop begins
Poll IOPS every SampleIntervalSeconds
Update peak if current IOPS exceeds stored peak
Calculate usage% = (current / peak) x 100
usage% >= ThresholdPercent AND cooldown elapsed -> Send Discord alert and record timestamp
usage% >= ThresholdPercent AND cooldown active -> Log suppressed alert
usage% < ThresholdPercent -> Log normal data only
Sleep SampleIntervalSeconds -> return to top of loop

8. Final Notes

The Hyper-V IOPS Monitor is designed to be a production-ready, low-maintenance solution. Once deployed, it requires no manual threshold tuning, adapts to workload growth automatically, and restarts itself in the event of an unexpected failure.

Recommended Next Steps

1. Adjust -BaselineDurationMinutes to match the time your VMs typically take to reach full load after a reboot. 2. Tune -AlertCooldownMinutes based on the volume of notifications your team finds manageable. 3. Consider modifying the Discord embed to include a hyperlink to your monitoring dashboard for faster incident response. 4. Review the log file after the first 24 hours of operation to confirm that learned peaks reflect realistic workloads.

For any issues, the log file at C:\Scripts\HyperV-IOPS-Monitor.log is the first place to look. Every poll cycle, every peak update, every alert attempt, and every error is recorded there with a precise timestamp.

— End of Document —